

Go Runtime & Goroutine Architecture Roadmap

Goal

Understand how Go can efficiently run hundreds of thousands or even millions of goroutines using a small number of operating system threads.

Level 0: Operating System Fundamentals

Before understanding Go, understand what the OS provides.

Process

A process is a running program.

Example:

- Chrome
- VS Code
- Go Application

Each process has:

- Memory
- Resources
- Threads

Example:

Process

```
|— Thread 1
|— Thread 2
|— Thread 3
```

Thread

A thread is the unit scheduled by the operating system.

The CPU executes threads directly.

Important:

Goroutines are NOT threads.

Go builds goroutines on top of threads.

Level 1: Why Goroutines Exist

Creating threads is expensive.

Typical OS Thread:

- ~1 MB stack
- OS scheduling overhead
- Expensive context switching

Example:

10,000 Threads
≈ 10 GB Memory

Not practical.

Go introduces:

Goroutines

Lightweight concurrent execution units.

Initial stack size:

~2 KB

Benefits:

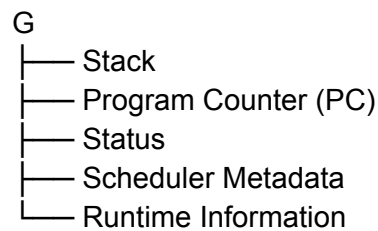
- Very small memory footprint

- Fast creation
 - Millions can exist simultaneously
-

Level 2: Goroutine (G)

The most important runtime structure.

A Goroutine contains:



Stack

Stores:

- Local variables
- Function arguments
- Return values

Example:

```
func add() {  
  x := 10  
  y := 20  
}
```

x and y live on the stack.

Program Counter (PC)

Tracks:

"What instruction should execute next?"

Example:

```
func worker() {  
    fmt.Println("A")  
    fmt.Println("B")  
    fmt.Println("C")  
}
```

If execution pauses after A:

PC remembers position.

When resumed:

Execution continues from B.

Goroutine States

A goroutine may be:

- Runnable
- Running
- Waiting
- Syscall
- Dead

Lifecycle:

Created

↓

Runnable

↓

Running

↓

Waiting

↓

Runnable

↓

Running

↓

Dead

Level 3: Machine (M)

M represents an Operating System Thread.

CPU

↓

M (OS Thread)

↓

G (Goroutine)

Without an M:

No goroutine can execute.

Level 4: Processor (P)

The most misunderstood component.

P is NOT:

- CPU
- Thread
- Goroutine

P is:

Scheduler Context

Contains:

- Local Run Queue
- Memory Cache
- Runtime Data
- Timer Information

Think:

G = Task
M = Worker
P = Work Desk

A worker needs a desk to perform work.

Execution requires:

M + P + G

Level 5: Local Run Queue

Each P owns its own queue.

Example:

P1
├── G1
├── G2
└── G3

P2
├── G4
└── G5

Runnable goroutines wait here.

The attached thread executes goroutines from this queue.

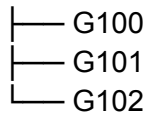
Level 6: Global Run Queue

Used when:

- Local queues overflow
- Work must be shared

Example:

Global Queue



Schedulers may pull work from here.

Level 7: Scheduler Loop

The heart of the runtime.

Conceptually:

```
while true {  
    find runnable goroutine  
  
    execute goroutine  
  
    if blocked:  
        save state  
  
    if queue empty:  
        steal work  
  
    if network ready:  
        wake goroutine  
  
    if running too long:  
        preempt  
  
}
```

The scheduler continuously manages goroutines.

Level 8: Context Switching

Switching execution between goroutines.

Example:

G1
pause

G2
pause

G1
resume

Runtime saves:

- Program Counter
- Stack Information
- Registers

This is much cheaper than OS thread switching.

Level 9: Work Stealing

Problem:

P1
├── G1
├── G2
├── G3
├── G4
└── G5

P2
└── Empty

One CPU works.

Another CPU sits idle.

Solution:

P2 steals roughly half of P1's work.

Result:

P1
├── G1
└── G2

P2
├── G3
├── G4
└── G5

All CPUs remain busy.

Level 10: Preemption

Problem:

```
func worker() {  
  for {  
  }  
}
```

Without preemption:

One goroutine can monopolize the CPU forever.

Runtime periodically checks:

"Has this goroutine run too long?"

If yes:

- Pause it
- Save its state
- Put it back into a queue
- Execute another goroutine

This guarantees fairness.

Level 11: Network Poller

One of the most important Go runtime components.

Example:

```
conn.Read()
```

If no data exists:

The goroutine enters Waiting state.

The runtime immediately frees:

- M
- P

for other work.

Meanwhile the Network Poller monitors sockets using:

Linux:

- epoll

macOS:

- kqueue

Windows:

- IOCP

When data arrives:

Network Poller

↓

Wake Goroutine

↓

Put Back Into Queue

↓

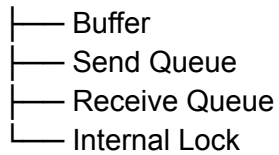
Continue Execution

This is how Go handles huge numbers of connections efficiently.

Level 12: Channel Internals

Channels contain:

Channel



Unbuffered Channels

Sender waits until receiver exists.

Example:

```
ch <- 10
```

Blocks until:

```
<-ch
```

receives.

Buffered Channels

Sender may continue immediately if buffer has space.

Benefits:

- Increased throughput
 - Reduced blocking
-

Level 13: Select Internals

Example:

```
select {  
case <-ch1:  
case <-ch2:  
}
```

Concepts:

- Channel selection
- Fairness
- Wake-up logic
- Scheduler interaction

Critical for backend systems.

Level 14: Synchronization Primitives

Mutex

Protects shared data.

sync.Mutex

RWMutex

Allows:

- Multiple readers
 - Single writer
-

WaitGroup

Wait for multiple goroutines.

Once

Run code exactly once.

Cond

Condition-based synchronization.

Sync.Map

Concurrent map implementation.

Atomic Operations

Lock-free synchronization.

Examples:

```
atomic.AddInt64()  
atomic.LoadInt64()
```

Level 15: Memory Model

Important concepts:

- Happens Before
- Visibility
- Ordering
- Synchronization

Questions answered:

Why can data races occur?

Why do mutexes work?

Why do channels synchronize memory?

Level 16: Stack Growth

Traditional Threads:

1 MB Fixed Stack

Go:

2 KB Initial Stack

Growth:

2 KB

↓

4 KB

↓

8 KB

↓

16 KB

↓

32 KB

Stacks grow dynamically.

This is a major reason millions of goroutines are possible.

Level 17: Heap vs Stack

Stack

Characteristics:

- Fast
- Per-goroutine

- Automatically cleaned
-

Heap

Characteristics:

- Shared memory
- Garbage collected
- More expensive

Understanding this is critical for performance optimization.

Level 18: Escape Analysis

Compiler optimization.

Example:

```
func test() *int {  
  x := 10  
  return &x  
}
```

x cannot remain on stack.

Compiler moves it to heap.

This is called:

Escape Analysis

Useful for understanding allocations.

Level 19: System Calls

Example:

os.Open()
syscall.Read()

Some system calls block.

Runtime may:

- Detach P
- Create another M

This prevents scheduler stalls.

Level 20: Garbage Collector

Go uses a concurrent garbage collector.

Important concepts:

- Mark Phase
- Sweep Phase
- Tri-color Marking
- Write Barriers
- Stop The World (STW)
- GC Pacing

Special GC Goroutines:

- Mark Workers
 - Sweep Workers
 - GC Assist
-

Level 21: Sysmon

A special runtime monitoring thread.

Responsibilities:

- Preemption
- Timer Management

- Network Monitoring
- GC Assistance
- Deadlock Detection

Think of Sysmon as:

"The Runtime Supervisor"

Level 22: Timers

Examples:

```
time.Sleep()  
time.After()  
time.NewTicker()
```

Runtime manages:

- Timer Heap
- Wakeups
- Scheduling Events

Used extensively in backend services.

Level 23: Scheduler Fairness

When thousands of goroutines exist:

How does Go ensure no goroutine starves?

Mechanisms:

- Local Queues
- Global Queue
- Work Stealing
- Preemption

Together they provide fair scheduling.

Level 24: Runtime Diagnostics

Essential for Production Engineers

Tools:

- pprof
- trace
- runtime package

Useful for analyzing:

- CPU usage
 - Memory usage
 - Scheduler behavior
 - Blocking operations
-

Level 25: Scheduler Tracing

Observe:

- Goroutine creation
- Scheduling decisions
- Work stealing
- Blocking events
- Wakeups
- Preemption

Provides visibility into runtime behavior.

Level 26: Goroutine Leaks

Example:

```
go func() {  
<-ch  
}()
```

If nobody sends:

The goroutine waits forever.

Result:

Memory leak
Resource leak

Production systems often suffer from goroutine leaks.

Level 27: Context Package

One of the most important backend concepts.

Example:

```
ctx, cancel := context.WithTimeout(...)
```

Features:

- Cancellation
- Deadlines
- Request Scope
- Propagation

Used in:

- HTTP servers
 - Databases
 - gRPC
 - Microservices
-

Complete Mental Model

When you write:

```
go worker()
```

The runtime roughly performs:

1. Create a Goroutine (G)
 2. Place G into a P's local queue
 3. M attached to P picks G
 4. CPU executes G
 5. If G blocks:
 - Save state
 - Free M and P
 6. Network Poller monitors readiness
 7. When ready:
 - Wake G
 - Put G back into queue
 8. Scheduler selects G again
 9. Continue execution
 10. Goroutine completes
 11. Runtime destroys G
-

Mastery Roadmap

Phase 1

- Process
- Thread
- Goroutine Lifecycle
- G-M-P

Phase 2

- Channels
- Select
- Mutex
- RWMutex
- WaitGroup
- Context

Phase 3

- Memory Model
- Atomic Operations
- Escape Analysis
- Heap vs Stack

Phase 4

- Network Poller
- Syscalls
- Timers
- Sysmon

Phase 5

- Garbage Collector
- Scheduler Tracing
- Runtime Diagnostics
- Performance Profiling

Mastering these topics provides a deep understanding of Go's concurrency model, runtime architecture, and production backend behavior.